EECS 492 - Homework 3 (Winter 2026)

Due: Wednesday, April 1, 2026 at 23:59

Logistics

Submission Guidelines

You must work alone for the conceptual problems, though you may discuss ideas and high-level concepts with classmates. You may optionally work with a partner on the coding sections, though you can work alone if you want.

The due date for this assignment is **April 1, 2026 at 23:59**. The submission is tracked using Gradescope. The written section and programming section are graded and penalized separately. Also note that any change you make to the homework already submitted on Gradescope counts as a resubmission. If you make any changes to the assignment after the due date has passed you will be assigned a late penalty based on the number of days that have passed. For example, if you edit an assignment on February 22 and it was due on February 19 you will be assigned a 30% penalty (10% per day). This is non-negotiable.

You must submit the following files to Gradescope (note that there are two separate Gradescope submissions):

1. A (legible) PDF containing the solutions to the written section. You can write your solutions by hand, use a tablet, or typeset your solutions in LaTeX. We only ask that you make it easy to read and not too verbose so that graders don't struggle to understand your writing. You must submit the written section to the Gradescope individually.
2. The completed `Agent.py` file to the programming section. If you work with a partner, you must put both people's names on the Gradescope submission.

Late Day Tokens

To accommodate for coinciding deadlines you may have from other courses, or personal unforeseen events such as sickness, each person has 5 late-day tokens for the whole semester (a maximum of 3 can be used for a particular homework). Each token provides an automatic extension of 1 calendar day—no questions asked.

1. Each late day token covers the whole homework (both written part and programming part) for extension of 1 calendar day. For example, if you submitted written part 1 day late and programming part 2 days late. Then, if you use 1 late day token, you don't have any penalty on the written part, but will have 10% penalty on the programming part.
2. Each homework has a Gradescope submission for the late day token, in addition to the written part and programming part.

3. Late days are rounded up to the nearest integer. For example, a submission that is 4 hours late will count as one day late.
4. Extreme circumstances, like medical emergencies, etc.: In such cases, additional, no-penalty extensions will be granted. Contact your section faculty instructor with some written documentation (like doctor's note).
5. Emails requesting exceptions for other reasons will be ignored with no reply.
6. Homework turned in after three days will not be accepted.

Grading Policy

You have one week to submit a regrade request once the grades for your homework are out. We will provide solutions, along with the exact grading rubric used.

Collaboration

Collaborating with people is encouraged, but plagiarism is strictly prohibited. As mentioned in **Submission Guidelines**, you must work alone for the conceptual problems, though you may discuss ideas and high-level concepts with classmates. You may optionally work with a partner on the coding sections, though you can work alone if you want.

Generative AI

Learning how to use AI functions such as ChatGPT is important for all of us. Used properly, ChatGPT can enhance our work; used improperly, it can border on plagiarism. You are allowed to use ChatGPT or similar only for conceptual questions, and **it is strictly forbidden for programming**. If you have used ChatGPT on anything you submit for EECS 492, please include an explanation as to:

- State which model you used
- Include the prompt(s) that you gave the model
- Explain your reasoning as to why you agree (or disagree) with the response
- Mention any incorrect responses or erroneous behavior that you may have seen

1 Written [50 Points]

1.1 Hill Climbing [15 Points]

Your friend guards each of their files with a 2-digit PIN code, but since they keep forgetting it, they decide to implement an AI agent that uses hill climbing to find it automatically. Each PIN comprises two integers, P_1 and P_2, each of which can take values from 0 to 9. They implement the following agent - the agent automatically submits a candidate PIN, and if it's correct, the file unlocks. If the guessed PIN is incorrect, the agent receives a score that measures how good the guess was. If the two digits entered are x and y, then the score returned is

$$h(x, y) = -(|x - P_1| + |y - P_2|)$$

The agent then uses a hill-climbing algorithm to attempt to find the PIN. The algorithm generates neighbors by creating all guesses that are different by one digit. In other words, if the current state is (x, y), the neighbors generated are $(x-1, y), (x+1, y), (x, y-1)$, and $(x, y+1)$. The algorithm doesn't generate any states outside the valid (0,0) to (9,9) range. In the event of a tie in the h value, the state with the lowest x value is selected.

(a) Assume that the initial guess for the PIN is (5,5) and that the actual PIN is (7,2). List the steps taken by the hill-climbing algorithm till it terminates. If it does not, explain why. (6 Points)

(b) Is this algorithm guaranteed to find the correct PIN code? Feel free to use a graphing tool like GeoGebra to help you solve this. (3 Points)

(c) Now assume that the algorithm is hard-coded to terminate after n steps. Does your answer to part (b) change if

  (i) $n = 10$ (2 Points)

  (ii) $n = 20$ (2 Points)

(d) Suppose we modify the heuristic a bit so that it's now

$$h(x, y) = |x - P_1| - |y - P_2|$$

Is this version of his hill-climbing search guaranteed to find a solution? Explain your answer. (2 Points)

1.2 Adversarial Search [10 Points]

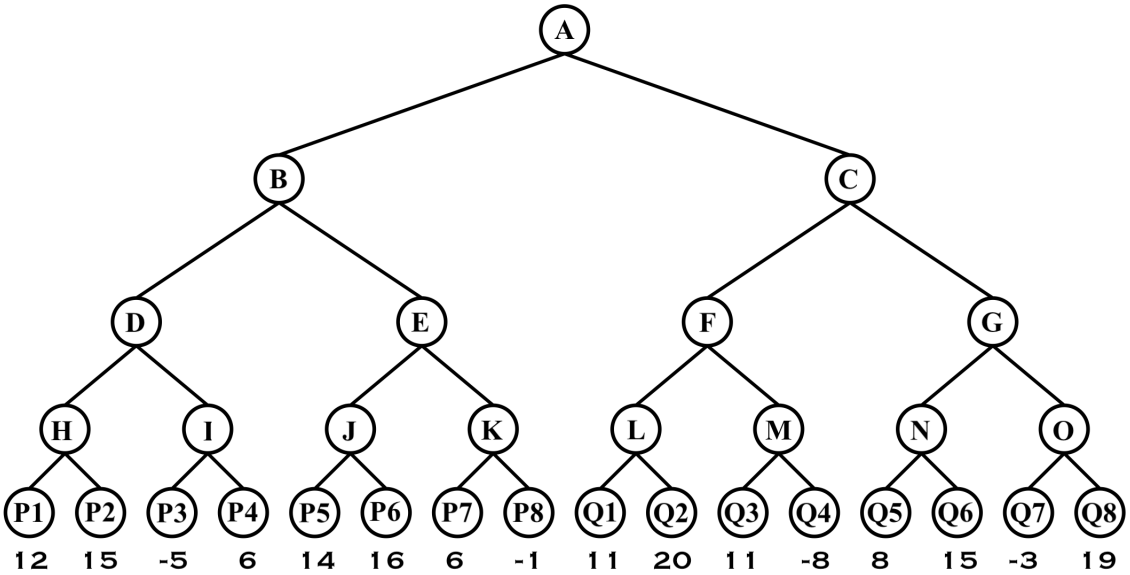


Figure 1: Game tree for a 2-player game.

(a) Figure 1 shows the game tree for a 2-player game. Apply **minimax (not alpha-beta pruning)** to this game tree to find the value of the root node. Please provide the values for each node in the table below. (4 Points)

Node	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Value															

(b) Apply α-β pruning to this tree (left to right) and list all the nodes that would be pruned and the **node parent's alpha and beta values**. For example, if you decide to prune node Q1, report the alpha and beta values for node L. If you decide to prune M (and therefore it's children Q3 and Q4), report the values for node F for M and leave them blank for Q3 and Q4. **Make sure to list all the nodes that can be pruned, including all the children nodes.** (6 Points)

1.3 Game Theory [9 Points]

You want to buy a whiteboard with your friend that costs \$4. Each of you only have \$3. You decide to pool the money together, and each person can contribute \$1, \$2, or \$3. You both hold the same principle: it's good to purchase the whiteboard by pooling your money together, but it's better to contribute less money to it.

(a) Construct the payoff matrix for this game. The payoff for spending d dollars is calculated by the function f, where:

$$f(d) = \begin{cases} 0 - d & \text{if the team } \textbf{doesn't} \text{ get the whiteboard} \\ 4 - d & \text{if the team } \textbf{does} \text{ get the whiteboard} \end{cases}$$

Use the payoff function f to construct the payoff matrix. (5 Points)

(b) Find the pure strategy Nash equilibrium for this game, if it exists. If there are multiple, list them all. If there are no equilibria, justify why. (2 Points)

(c) Is this game a zero-sum game? Justify why. (2 Points)

1.4 Constraint Satisfaction [16 Points]

You are planning to set up a full course meal for your Valentine's date. You have looked through the food options and decided on four restaurants. To ensure a successful date, you need to pick up 6 courses:

- Course 1 (C1): hors d'oeuvres
- Course 2 (C2): soup
- Course 3 (C3): appetizer
- Course 4 (C4): salad
- Course 5 (C5): main course
- Course 6 (C6): dessert

Courses are picked up in order, **with the exception of soup and the main course**. Since you don't want your soup and the main course to get cold, you will pick them up last in two restaurants that are next to each other. **You want to pick up the soup before you pick up the main course.**

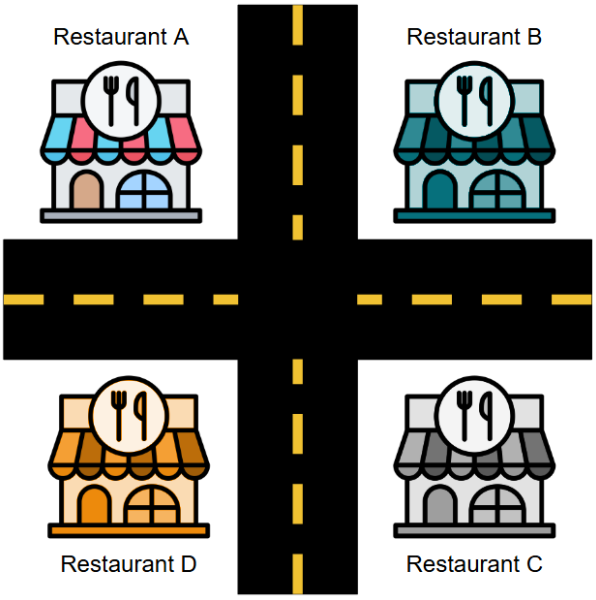


Figure 2: The layout of Restaurants A, B, C, and D around the block.

As depicted in Figure 2, each restaurant is located on the corners of the same block (so, restaurant A is next to restaurant B, B to C, C to D, and D to A). They offer different food options:

1. Restaurant A offers C2, C4, C5, C6
2. Restaurant B offers C1, C2, C3, C6
3. Restaurant C offers C1, C2, C4, C5
4. Restaurant D offers C1, C4, C6

However, since the restaurants are short on staff, you can only pick up one course at a time. **You cannot pick up two sequential courses from the same restaurant. (Sequential here refers to the order that the courses are picked up in, *not* the numerical order of the courses).** You also want to pick up dessert at the same restaurant as hors d'oeuvres so you can have a consistent start and end to the date.

(a) Make this scenario into a CSP where each task is assigned a variable, and list out the domains and constraints for each variable. You may need more or less rows than are currently included in this table. (6 Points)

Variable	Domain	Constraints

(b) Draw the constraint graph. (1 Points)

(c) What variable would be assigned first if you use the minimum remaining value heuristic? (1 Point)

(d) What variable would be assigned first if you use the degree heuristic? (1 Point)

(e) Step through AC-3 by hand using the constraint graph you drew in part (b), and list the updated domains for each variable. You may need more or less rows than are currently included in this table. (6 Points)

Variable	Domain

(f) Now, we want to introduce a new constraint: we must visit each of the restaurants at least once. What is a solution to this new CSP? If there is more than one answer, list all of them. (1 Points)

2 Programming [50 Points]

2.1 Setup

All programming assignments for this course use Python. If you're new to Python or not very experienced with it, please come to office hours; we're happy to help! We recommend also checking out the Python Cheatsheet. The topics you'd need from here are: Basics, Built-in Functions, Control Flow, Functions, Lists, Tuples, Dictionaries, Sets, OOP, and Virtual Environments. We recommend also checking out the Python Resources document on Canvas. We have also shared a document Debugging Tips to help you debug your code. These resources are on Canvas, under 'Files' > 'Coding Resources'.

We highly recommend creating a new virtual environment for the assignment. You can then simply install the requirements by running the command
`python -m pip install -r requirements.txt`

2.2 Problem Statement

For this programming section, you will be working on creating a Tic-Tac-Toe (TTT) AI and Connect-4 (C4) AI. You will be using minimax with $\alpha\beta$ pruning in `Agent.py` to create an agent that will predict the best set of moves to maximize the chances of winning in either game. If you are not familiar with the rules of these games, you can learn about the rules for TTT and C4 here.

The assignment folder contains these files:

- `Agent.py`
- `Game.py`
- `TTT.py`
- `Connect4.py`
- `requirements.txt`

Complete the implementation of the `minimax` function in the given `Agent.py` file and submit `Agent.py` to Gradescope. You will be evaluated using the Autograder on 10 test C4 games.

Please go through the starter code files—especially `Game.py`—before you begin! Each of the starter code files are described in the following subsections. You will never need to modify `Game.py`, `TTT.py`, or `C4.py`.

Agent.py

The relevant function here is:

- `minimax(self, board, depth, maximizingPlayer, alpha, beta)`: this is a recursive function for which you must complete the implementation. You must complete this function using $\alpha\beta$ pruning, as regular minimax will take too long to run. When implementing the function, it is important to **break ties by prioritizing winning in the fewest moves (losing with as many moves as possible)**. The output of this function is the move made and the score.

Game.py

This file references the two games we have implemented for you, Tic-Tac-Toe and Connect 4, and this is where you will test your implementation with the tests. An important variable is:

- `turn`: this variable decides whose turn it is, the human or AI. It is randomly initialized to one of the players in the main method. However, if you are first testing your implementation, you may want to initialize it to a specific player for consistent results.

TTT.py and Connect4.py

Both files implement their respective games and share important function names for easy implementation of the `minimax` function. You should not modify these functions, but familiarize yourself with them, as they might be useful in writing `minimax`.

Important functions are:

- `get_valid_moves(board)`: returns list of columns/coordinates where a move can be made based on current board.
- `play_move(board, move, player)`: makes a deep copy of the board and returns the updated board with the new move made by a player.
- `heuristic_value(board, player)`: calculates and returns the score for a player based on that player's moves.
- `is_winner(board, player)`: boolean function that determines if either player has put their game piece in 3 consecutive cells for TTT, or 4 consecutive cells for Connect 4
- `is_full(board)`: boolean function that returns if the board is full

One of the main differences in the games is how you view them. Since TTT is quite basic, it can be played in your terminal. Since Connect 4 is a 6x7 grid, we have included code to simulate the game in another window to better visualize the board.

2.3 Testing locally

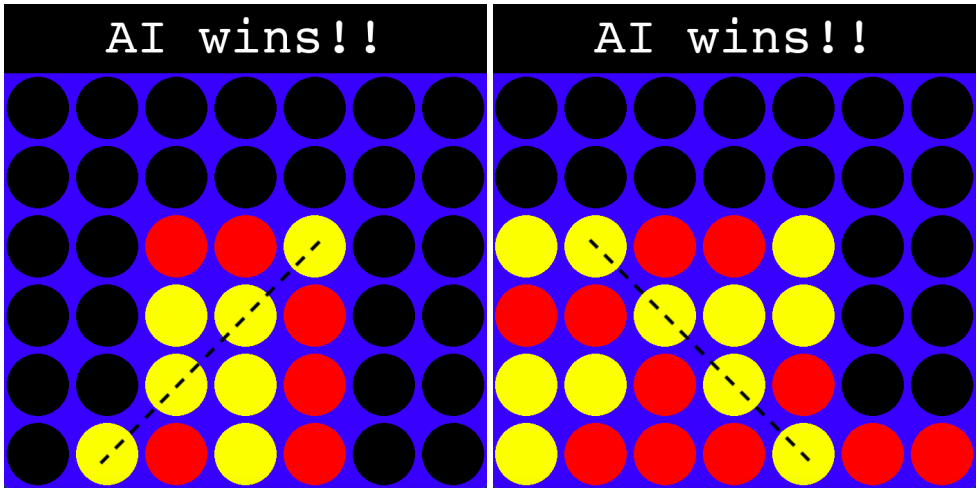
We have provided four test cases for Connect 4 that you can use to test your implementation. Since Tic-Tac-Toe does not have many solutions, you can test its correctness using the terminal or make your own tests.

To try your code without tests, you can run
`python Game.py TTT` or `python Game.py C4`.

To try the test cases for C4, you can run
`python Game.py C4 test[n].txt` where n is the test number from 0 to 3.

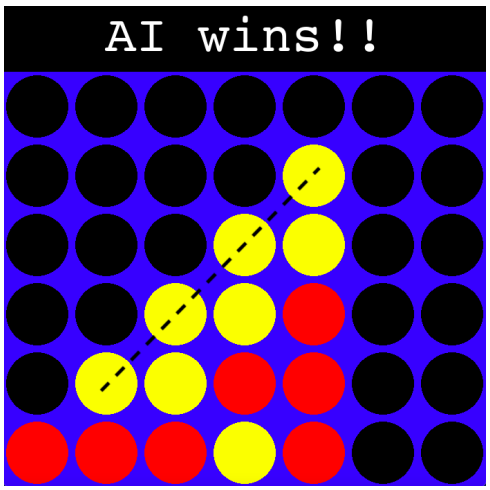
In `expected_results.txt`, you will see the columns you need to make your move as the human, how many moves it should take the agent to win, and the winning combination stated by its row and column location. **The given test cases are identical to the comparably named tests on Gradescope.** There are no hidden test cases; your score on Gradescope is the score you can expect to receive for this section.

To better visualize the local tests, you can view the final board state for each test below in Figure 3. The dotted line is the winning combination by the agent.

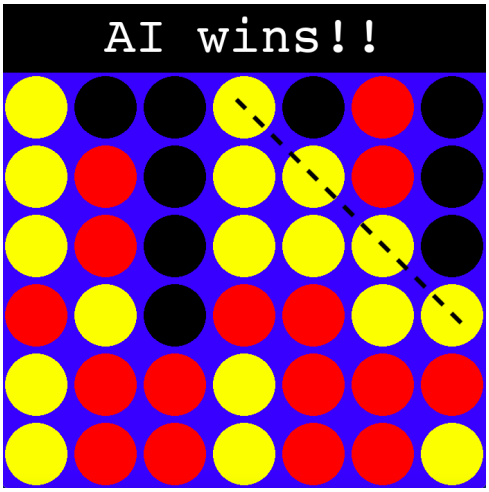


(a) Test 0

(b) Test 1



(c) Test 2



(d) Test 3

Figure 3: Final board states for the local tests